

Complete Template Library

Fourier Series & Continuous Fourier Transform

CSE 220 — Signals and Linear Systems

Topics Covered:

1D CFT · IFT · Differentiation Property
Time-Shift Property · Scaling & Modulation
Fourier Series · Fourier Epicycles
2D CFT · 2D IFT · Edge Detection
Image Denoising · Parseval's Theorem
Plotting Templates · MSE Analysis
Hidden Frequency Discovery

Contents

1	UNIVERSAL BUILDING BLOCKS	3
1.1	Template 1: Imports and Axes Setup	3
1.2	Template 2: 1D CFT Function	3
1.3	Template 3: 1D IFT Function	3
1.4	Template 4: MSE Function	4
1.5	Template 5: Signal Definitions	4
2	PLOTTING TEMPLATES	5
2.1	Template 6: Basic 2-Column Plot	5
2.2	Template 7: 2x2 Comparison Plot	5
2.3	Template 8: Overlap Verification Plot	6
2.4	Template 9: 3-Row Plot (All 3 Derivatives)	7
2.5	Template 10: Image Display Plot	7
3	PROBLEM 1: DIFFERENTIATION PROPERTY	8
3.1	Template 11: Full Differentiation Property Solution	8
4	PROBLEM 2: TIME-SHIFT PROPERTY	9
4.1	Template 12: SignalGenerator Class	10
4.2	Template 13: CFTAnalyzer Class	10
4.3	Template 14: Full Time-Shift Solution	11
5	PROBLEM 3: SCALING & MODULATION PROPERTY	12
5.1	Template 15: Scaling + Modulation Full Solution	12
6	PROBLEM 4: HIDDEN FREQUENCY DISCOVERY	14
6.1	Template 16: Frequency Discovery Full Solution	14
7	PROBLEM 5: PARSEVAL'S THEOREM	16
7.1	Template 17: Parseval's Full Solution	16
8	OFFLINE 1: FOURIER EPICYCLES (FS REDRAWER)	18
8.1	Template 18: Complete FourierEpicycles Class	18
9	OFFLINE 2: 2D CFT EDGE DETECTOR	20
9.1	Template 19: ContinuousImage Class	20
9.2	Template 20: CFT2D Class — compute_cft()	20
9.3	Template 21: InverseCFT2D Class	22
9.4	Template 22: Complete Edge Detection Pipeline	23
10	PROBLEM 6: IMAGE DENOISING	23
10.1	Template 23: Image Denoising Full Solution	24
11	QUICK REFERENCE CARDS	25
11.1	Template 24: CFT vs IFT vs Fourier Series	25
11.2	Template 25: All Properties Summary	26
11.3	Template 26: MSE Interpretation	26

11.4	Template 27: 2D CFT vs 2D IFT Summary	26
11.5	Template 28: Common Mistakes	27

1 UNIVERSAL BUILDING BLOCKS

1.1 Template 1: Imports and Axes Setup

Always Start With This

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from imageio.v2 import imread # only if working with images
4
5 #           Time and Frequency Axes
6 t = np.linspace(-10, 10, 5000) # time axis
7 f = np.linspace(-5, 5, 1000)   # frequency axis
8
9 #           For image problems
10 t = np.linspace(-5, 5, 2000)   # problem specific
11 f = np.linspace(-10, 10, 1000) # problem specific

```

1.2 Template 2: 1D CFT Function

1D CFT — Use This Everywhere

```

1 def compute_cft(signal, t, f):
2     """
3     CFT:  $X(f) = \text{integral}[x(t) * e^{-j2\pi f t}] dt$ 
4      $e^{-j2\pi f t} = \cos(2\pi f t) - j\sin(2\pi f t)$ 
5     Use trapezoid for integration. Never use np.fft!
6     """
7     X = np.zeros(len(f), dtype=complex)
8     for i, fi in enumerate(f):
9         kernel = np.cos(2*np.pi*fi*t) - 1j*np.sin(2*np.pi*fi*t)
10        X[i] = np.trapezoid(signal * kernel, t)
11    return X

```

1.3 Template 3: 1D IFT Function

1D IFT — Positive Sign Version

```

1 def compute_ifft(X, f, t):
2     """
3     IFT:  $x(t) = \text{integral}[X(f) * e^{+j2\pi f t}] df$ 
4     KEY DIFFERENCE from CFT: POSITIVE sign in exponent!
5     Forward CFT:  $e^{-j2\pi f t}$  [negative]
6     Inverse IFT:  $e^{+j2\pi f t}$  [positive]
7     """
8     x_reconstructed = np.zeros(len(t), dtype=complex)
9     for i, ti in enumerate(t):
10        kernel = np.cos(2*np.pi*f*ti) \
11        + 1j*np.sin(2*np.pi*f*ti) # POSITIVE

```

```

12         x_reconstructed[i] = np.trapezoid(X * kernel, f)
13     return np.real(x_reconstructed)

```

1.4 Template 4: MSE Function

MSE — Used in All Problems

```

1  def mse(a, b):
2      """
3      MSE = (1/N) * sum( |a - b|^2 )
4      Small MSE means a and b are nearly identical.
5      Use for both magnitude and phase comparisons.
6      """
7      return np.mean(np.abs(a - b)**2)
8
9  # Usage examples:
10 mse_magnitude = mse(np.abs(Y_direct), np.abs(Y_theory))
11 mse_phase     = mse(np.angle(Y_direct), np.angle(Y_theory))
12
13 print(f"Magnitude MSE : {mse_magnitude:.4e}")
14 print(f"Phase MSE     : {mse_phase:.4e}")

```

1.5 Template 5: Signal Definitions

Common Signals

```

1  # Gaussian
2  def gaussian(t, a=1):
3      return np.exp(-a * t**2)          # e^(-at^2)
4
5  # Square Wave
6  def square(t):
7      return np.where(np.abs(t) <= 0.5, 1.0, 0.0)
8
9  # Triangle Wave
10 def triangle(t):
11     return np.where(np.abs(t) <= 1.0, 1.0 - np.abs(t), 0.0)
12
13 # Cosine/Sine
14 x = 0.5*np.cos(4*t) + 0.5*np.sin(6*t)
15
16 # Piecewise (parabolas)
17 def piecewise(t):
18     return np.where(
19         np.abs(t) <= 1,
20         1 - t**2,          # middle parabola
21         np.where(
22             np.abs(t) <= 3,
23             (np.abs(t) - 3)**2,  # side parabolas
24             0.0                # outside = zero
25         )

```

```

26     )
27
28     #           Complicated signal (simplifies to 3 sine waves)
29     ft = 2*np.sin(14*np.pi*t) \
30         - np.sin(2*np.pi*t)*(4*np.sin(2*np.pi*t)*np.sin(14*np.pi*t)
          -1)

```

2 PLOTTING TEMPLATES

2.1 Template 6: Basic 2-Column Plot

Magnitude and Phase Side by Side

```

1  fig, axes = plt.subplots(1, 2, figsize=(12, 5))
2  fig.suptitle("CFT Analysis", fontsize=13, fontweight='bold')
3
4  # Magnitude
5  axes[0].plot(f, np.abs(X_f), 'b-', lw=2, label='|X(f)|')
6  axes[0].set_title('Magnitude Spectrum')
7  axes[0].set_xlabel('Frequency f (Hz)')
8  axes[0].set_ylabel('|X(f)|')
9  axes[0].legend()
10 axes[0].grid(True, alpha=0.3)
11
12 # Phase
13 axes[1].plot(f, np.angle(X_f), 'r-', lw=2, label='Phase X(f)')
14 axes[1].set_title('Phase Spectrum')
15 axes[1].set_xlabel('Frequency f (Hz)')
16 axes[1].set_ylabel('Phase (radians)')
17 axes[1].legend()
18 axes[1].grid(True, alpha=0.3)
19
20 plt.tight_layout()
21 plt.show()

```

2.2 Template 7: 2x2 Comparison Plot

Direct vs Theory: 2x2 Grid

```

1  fig, axes = plt.subplots(2, 2, figsize=(13, 9))
2  fig.suptitle("Property Verification", fontsize=13, fontweight='
    bold')
3
4  # Top-left: Magnitude direct
5  axes[0,0].plot(f, np.abs(Y_direct), 'b-', lw=2, label='Direct')
6  axes[0,0].set_title('|Y(f)| Direct')
7  axes[0,0].set_xlabel('f (Hz)'); axes[0,0].set_ylabel('Magnitude')
8  axes[0,0].legend(); axes[0,0].grid(True, alpha=0.3)
9

```

```

10 # Top-right: Magnitude theory
11 axes[0,1].plot(f, np.abs(Y_theory), 'r--', lw=2, label='Theory')
12 axes[0,1].set_title('|Y(f)| Theory')
13 axes[0,1].set_xlabel('f (Hz)'); axes[0,1].set_ylabel('Magnitude'
14 )
15 axes[0,1].legend(); axes[0,1].grid(True, alpha=0.3)
16
17 # Bottom-left: Phase direct
18 axes[1,0].plot(f, np.angle(Y_direct), 'b-', lw=2, label='Direct
19 ')
20 axes[1,0].set_title('Phase Direct')
21 axes[1,0].set_xlabel('f (Hz)'); axes[1,0].set_ylabel('Phase (rad
22 )')
23 axes[1,0].legend(); axes[1,0].grid(True, alpha=0.3)
24
25 # Bottom-right: Phase theory
26 axes[1,1].plot(f, np.angle(Y_theory), 'r--', lw=2, label='Theory
27 ')
28 axes[1,1].set_title('Phase Theory')
29 axes[1,1].set_xlabel('f (Hz)'); axes[1,1].set_ylabel('Phase (rad
30 )')
31 axes[1,1].legend(); axes[1,1].grid(True, alpha=0.3)
32
33 plt.tight_layout()
34 plt.show()

```

2.3 Template 8: Overlap Verification Plot

Overlap Plot — Direct vs Theory on Same Axes

```

1 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
2 fig.suptitle("Overlap Verification (Blue=Direct, Red=Theory)",
3             fontsize=13)
4
5 # Magnitude overlap
6 axes[0].plot(f, np.abs(Y_direct), 'b-', lw=3,
7             label='Direct', alpha=0.7)
8 axes[0].plot(f, np.abs(Y_theory), 'r--', lw=1.5,
9             label='Theory')
10 axes[0].set_title(f'Magnitude Overlap (MSE={mse_mag:.2e})')
11 axes[0].set_xlabel('f (Hz)')
12 axes[0].set_ylabel('Magnitude')
13 axes[0].legend(); axes[0].grid(True, alpha=0.3)
14
15 # Phase overlap
16 axes[1].plot(f, np.angle(Y_direct), 'b-', lw=3,
17             label='Direct', alpha=0.7)
18 axes[1].plot(f, np.angle(Y_theory), 'r--', lw=1.5,
19             label='Theory')
20 axes[1].set_title(f'Phase Overlap (MSE={mse_phase:.2e})')
21 axes[1].set_xlabel('f (Hz)')
22 axes[1].set_ylabel('Phase (rad)')
23 axes[1].legend(); axes[1].grid(True, alpha=0.3)
24

```

```

25 plt.tight_layout()
26 plt.show()

```

2.4 Template 9: 3-Row Plot (All 3 Derivatives)

3 Rows x 2 Cols — For Derivative Problems

```

1  fig, axes = plt.subplots(3, 2, figsize=(12, 12))
2  fig.suptitle("Differentiation Property      All 3 Derivatives")
3
4  pairs = [
5      (Y1_f, Y1_theory, "1st Derivative"),
6      (Y2_f, Y2_theory, "2nd Derivative"),
7      (Y3_f, Y3_theory, "3rd Derivative"),
8  ]
9
10 for row, (direct, theory, title) in enumerate(pairs):
11
12     # Magnitude
13     axes[row,0].plot(f, np.abs(direct), 'b-', lw=2.5,
14                     label='Direct', alpha=0.7)
15     axes[row,0].plot(f, np.abs(theory), 'r--', lw=1.5,
16                     label='Theory')
17     axes[row,0].set_title(f'{title}      Magnitude')
18     axes[row,0].set_xlabel('f (Hz)')
19     axes[row,0].set_ylabel('|Y(f)|')
20     axes[row,0].legend(); axes[row,0].grid(True, alpha=0.3)
21
22     # Phase
23     axes[row,1].plot(f, np.angle(direct), 'b-', lw=2.5,
24                     label='Direct', alpha=0.7)
25     axes[row,1].plot(f, np.angle(theory), 'r--', lw=1.5,
26                     label='Theory')
27     axes[row,1].set_title(f'{title}      Phase')
28     axes[row,1].set_xlabel('f (Hz)')
29     axes[row,1].set_ylabel('Phase (rad)')
30     axes[row,1].legend(); axes[row,1].grid(True, alpha=0.3)
31
32 plt.tight_layout()
33 plt.show()

```

2.5 Template 10: Image Display Plot

Image Plots (Grayscale)

```

1  fig, axes = plt.subplots(1, 3, figsize=(15, 5))
2
3  # Original image
4  axes[0].imshow(original_image, cmap='gray')
5  axes[0].set_title('Original Image')
6  axes[0].axis('off')

```

```

7
8 # Magnitude spectrum (log scale for visibility)
9 magnitude = np.sqrt(real**2 + imag**2)
10 axes[1].imshow(np.log(1 + magnitude), cmap='gray')
11 axes[1].set_title('Log Magnitude Spectrum')
12 axes[1].axis('off')
13
14 # Reconstructed / edge map
15 axes[2].imshow(result_image, cmap='gray')
16 axes[2].set_title('Result')
17 axes[2].axis('off')
18
19 plt.tight_layout()
20 plt.show()

```

3 PROBLEM 1: DIFFERENTIATION PROPERTY

Formula

$$\mathcal{F}\left\{\frac{d^n}{dt^n}x(t)\right\} = (j2\pi f)^n \cdot X(f)$$

3.1 Template 11: Full Differentiation Property Solution

Complete Template

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Setup
5 t = np.linspace(-10, 10, 5000)
6 f = np.linspace(-5, 5, 1000)
7
8 # Signals (manual derivatives)
9 x = 0.5*np.cos(4*t) + 0.5*np.sin(6*t) # original
10 y1 = -2.0*np.sin(4*t) + 3.0*np.cos(6*t) # dx/dt
11 y2 = -8.0*np.cos(4*t) - 18.0*np.sin(6*t) # d2x/dt2
12 y3 = 32.0*np.sin(4*t) - 108.0*np.cos(6*t) # d3x/dt3
13 # Rule: d/dt[cos(wt)] = -w*sin(wt)
14 # d/dt[sin(wt)] = w*cos(wt)
15
16 # CFT function
17 def cft(signal, t, f):
18     X = np.zeros(len(f), dtype=complex)
19     for i, fi in enumerate(f):
20         kernel = np.cos(2*np.pi*fi*t) - 1j*np.sin(2*np.pi*fi*t)
21         X[i] = np.trapezoid(signal * kernel, t)
22     return X
23
24 # Direct CFTs
25 X_f = cft(x, t, f)

```

```

26 Y1_f = cft(y1, t, f)
27 Y2_f = cft(y2, t, f)
28 Y3_f = cft(y3, t, f)
29
30 #           Theoretical: multiply by (j2*pi*f)^n
31 j2pif      = 1j * 2 * np.pi * f
32 Y1_theory  = j2pif      * X_f      # (j2*pi*f)^1
33 Y2_theory  = j2pif**2 * X_f      # (j2*pi*f)^2
34 Y3_theory  = j2pif**3 * X_f      # (j2*pi*f)^3
35
36 #           MSE
37 def mse(a, b): return np.mean(np.abs(a - b)**2)
38
39 print("==== MSE RESULTS =====")
40 for name, direct, theory in [
41     ("1st", Y1_f, Y1_theory),
42     ("2nd", Y2_f, Y2_theory),
43     ("3rd", Y3_f, Y3_theory)]:
44     print(f"{name}: Mag={mse(np.abs(direct), np.abs(theory)):.4e}
45           "
46           f"Phase={mse(np.angle(direct), np.angle(theory)):.4e}")
47
48 #           Plot (3x2 grid)
49 fig, axes = plt.subplots(3, 2, figsize=(12, 12))
50 pairs = [(Y1_f, Y1_theory, "1st"), (Y2_f, Y2_theory, "2nd"),
51          (Y3_f, Y3_theory, "3rd")]
52 for row, (direct, theory, name) in enumerate(pairs):
53     axes[row, 0].plot(f, np.abs(direct), 'b-', lw=2.5, alpha=0.7,
54                      label='Direct')
55     axes[row, 0].plot(f, np.abs(theory), 'r--', lw=1.5, label='
56         Theory')
57     axes[row, 0].set_title(f'{name} Derivative Magnitude')
58     axes[row, 0].legend(); axes[row, 0].grid(True, alpha=0.3)
59     axes[row, 1].plot(f, np.angle(direct), 'b-', lw=2.5, alpha
60                      =0.7,
61                      label='Direct')
62     axes[row, 1].plot(f, np.angle(theory), 'r--', lw=1.5, label='
63         Theory')
64     axes[row, 1].set_title(f'{name} Derivative Phase')
65     axes[row, 1].legend(); axes[row, 1].grid(True, alpha=0.3)
66 plt.tight_layout(); plt.show()

```

4 PROBLEM 2: TIME-SHIFT PROPERTY

Formula

$$\mathcal{F}\{x(t - t_0)\} = X(f) \cdot e^{-j2\pi f t_0}$$

$$|Y(f)| = |X(f)| \quad (\text{magnitude unchanged})$$

$$\angle Y(f) = \angle X(f) - 2\pi f t_0 \quad (\text{phase shifts linearly})$$

4.1 Template 12: SignalGenerator Class

OOP: SignalGenerator

```

1 class SignalGenerator:
2     def __init__(self, t):
3         self.t = t
4
5     def gaussian(self, a):
6         """x(t) = e-a*t2"""
7         return np.exp(-a * self.t**2)
8
9     def cosine(self, f0, A=1):
10        """x(t) = A*cos(2*pi*f0*t)"""
11        return A * np.cos(2*np.pi*f0*self.t)
12
13    def sine(self, f0, A=1):
14        """x(t) = A*sin(2*pi*f0*t)"""
15        return A * np.sin(2*np.pi*f0*self.t)
16
17    def square(self):
18        """Square wave: 1 if |t|<=0.5 else 0"""
19        return np.where(np.abs(self.t) <= 0.5, 1.0, 0.0)
20
21    def triangle(self):
22        """Triangle wave: 1-|t| if |t|<=1 else 0"""
23        return np.where(np.abs(self.t) <= 1.0,
24                        1.0 - np.abs(self.t), 0.0)
25
26    def time_shift(self, signal_fn, a, t0):
27        """
28        y(t) = x(t - t0)
29        Replace t with (t - t0) in signal function.
30        Do NOT manually shift arrays.
31        """
32        original_t = self.t
33        self.t = original_t - t0      # shift the time axis
34        shifted = signal_fn(a)       # evaluate at shifted
35                                     time
36        self.t = original_t          # restore original
37        return shifted

```

4.2 Template 13: CFTAnalyzer Class

OOP: CFTAnalyzer

```

1 class CFTAnalyzer:
2     def __init__(self, t, f):
3         self.t = t
4         self.f = f
5
6     def compute(self, signal):
7         """CFT using trapezoid integration."""
8         X = np.zeros(len(self.f), dtype=complex)

```

```

9         for i, fi in enumerate(self.f):
10             kernel = np.cos(2*np.pi*fi*self.t) \
11                 - 1j*np.sin(2*np.pi*fi*self.t)
12             X[i] = np.trapezoid(signal * kernel, self.t)
13         return X
14
15     def magnitude(self, X):
16         return np.abs(X)
17
18     def phase(self, X):
19         return np.angle(X)
20
21     def mse(self, a, b):
22         return np.mean((a - b)**2)

```

4.3 Template 14: Full Time-Shift Solution

Complete Time-Shift Template

```

1  t = np.linspace(-5, 5, 2000)
2  f = np.linspace(-10, 10, 1000)
3  t0 = 1 # time shift amount
4
5  gen = SignalGenerator(t)
6  x = gen.gaussian(a=1) # original signal
7  y = gen.time_shift(gen.gaussian, a=1, t0=t0) # shifted
8
9  analyzer = CFTAnalyzer(t, f)
10 X_f = analyzer.compute(x)
11 Y_f = analyzer.compute(y)
12
13 # Verify Property
14 mag_X = analyzer.magnitude(X_f)
15 mag_Y = analyzer.magnitude(Y_f)
16 phase_X = analyzer.phase(X_f)
17 phase_Y = analyzer.phase(Y_f)
18 phase_theory = phase_X - 2*np.pi*f*t0 # theoretical
    prediction
19
20 mse_mag = analyzer.mse(mag_X, mag_Y)
21 mse_phase = analyzer.mse(phase_Y, phase_theory)
22 print(f"Magnitude MSE : {mse_mag:.6e}")
23 print(f"Phase MSE : {mse_phase:.6e}")
24
25 # Plot
26 fig, axes = plt.subplots(2, 2, figsize=(13, 9))
27 axes[0,0].plot(f, mag_X, 'b--', lw=2, label='|X(f)|')
28 axes[0,0].plot(f, mag_Y, 'r--', lw=1.5, label='|Y(f)|')
29 axes[0,0].set_title('Magnitude (should overlap)')
30 axes[0,0].legend(); axes[0,0].grid(True, alpha=0.3)
31
32 axes[0,1].plot(f, phase_X, 'b--', lw=2, label='Phase X(f)')
33 axes[0,1].plot(f, phase_Y, 'r--', lw=1.5, label='Phase Y(f)')
34 axes[0,1].plot(f, phase_theory, 'g:', lw=2, label='Theory')

```

```

35 axes[0,1].set_title('Phase Spectra')
36 axes[0,1].legend(); axes[0,1].grid(True, alpha=0.3)
37
38 axes[1,0].plot(t, x, 'b-', lw=2, label='x(t)')
39 axes[1,0].plot(t, y, 'r--', lw=1.5, label='y(t)=x(t-1)')
40 axes[1,0].set_title('Time Domain Signals')
41 axes[1,0].legend(); axes[1,0].grid(True, alpha=0.3)
42
43 phase_diff = phase_Y - phase_X
44 axes[1,1].plot(f, phase_diff, 'r-', lw=2, label='Measured diff')
45 axes[1,1].plot(f, -2*np.pi*f*t0, 'g--', lw=1.5, label='-2*pi*f*t0'
46 )
47 axes[1,1].set_title('Phase Difference (should be straight line)')
48 axes[1,1].legend(); axes[1,1].grid(True, alpha=0.3)
49 plt.tight_layout(); plt.show()

```

5 PROBLEM 3: SCALING & MODULATION PROPERTY

Formula

$$y(t) = x(at) \cdot e^{j2\pi f_0 t} \implies Y(f) = \frac{1}{|a|} X\left(\frac{f - f_0}{a}\right)$$

5.1 Template 15: Scaling + Modulation Full Solution

Complete Scaling + Modulation Template

```

1 class SignalGenerator:
2     def __init__(self, t):
3         self.t = t
4
5     def square(self):
6         return np.where(np.abs(self.t) <= 0.5, 1.0, 0.0)
7
8     def triangle(self):
9         return np.where(np.abs(self.t) <= 1.0,
10                        1.0 - np.abs(self.t), 0.0)
11
12     def combined(self):
13         return self.square() + self.triangle()
14
15     def apply_operations(self, f0, a):
16         """
17         y(t) = x(a*t) * e^(j*2*pi*f0*t)
18         Step 1: time compress (replace t with a*t)
19         Step 2: phase shift (multiply by complex exponential)
20         """

```

```

21         original_t    = self.t
22         self.t         = a * original_t           # compress time
23         x_compressed   = self.combined()
24         self.t         = original_t
25
26         phase_shift    = np.exp(1j * 2*np.pi * f0 * self.t)
27         return x_compressed * phase_shift         # complex signal!
28
29 class CFTAnalyzer:
30     def __init__(self, t, f):
31         self.t = t; self.f = f
32
33     def compute(self, signal):
34         X = np.zeros(len(self.f), dtype=complex)
35         for i, fi in enumerate(self.f):
36             kernel = np.cos(2*np.pi*fi*self.t) \
37                     - 1j*np.sin(2*np.pi*fi*self.t)
38             X[i] = np.trapezoid(signal * kernel, self.t)
39         return X
40
41     def interpolate(self, X, f_orig, f_new):
42         """Evaluate X at new frequency points using
43            interpolation."""
44         r = np.interp(f_new, f_orig, np.real(X), left=0, right
45                       =0)
46         i = np.interp(f_new, f_orig, np.imag(X), left=0, right
47                       =0)
48         return r + 1j*i
49
50 # Main
51 t = np.linspace(-5, 5, 2000)
52 f = np.linspace(-10, 10, 1000)
53 f0 = 10; a = 10
54
55 gen = SignalGenerator(t)
56 x = gen.combined()
57 y = gen.apply_operations(f0=f0, a=a)
58
59 analyzer = CFTAnalyzer(t, f)
60 X_f = analyzer.compute(x)
61 Y_f = analyzer.compute(y)
62
63 # Theory:  $Y(f) = (1/|a|) * X((f-f_0)/a)$ 
64 f_shifted = (f - f0) / a
65 X_shifted = analyzer.interpolate(X_f, f, f_shifted)
66 Y_theory = (1/np.abs(a)) * X_shifted
67
68 # MSE
69 mse_mag = np.mean((np.abs(Y_f) - np.abs(Y_theory))**2)
70 mse_phase = np.mean((np.angle(Y_f) - np.angle(Y_theory))**2)
71 print(f"Magnitude MSE: {mse_mag:.4e}")
72 print(f"Phase MSE : {mse_phase:.4e}")
73
74 # Plot
75 fig, axes = plt.subplots(2, 2, figsize=(13, 9))
76 axes[0,0].plot(f, np.abs(Y_f), 'b-', lw=2, label='|Y(f)| measured

```

```

    ')
74 axes[0,0].plot(f, np.abs(Y_theory), 'r--', lw=1.5, label='Theory')
75 axes[0,0].set_title('Magnitude Verification')
76 axes[0,0].legend(); axes[0,0].grid(True, alpha=0.3)
77
78 axes[0,1].plot(f, np.angle(Y_f), 'b-', lw=2, label='Phase Y(f)')
79 axes[0,1].plot(f, np.angle(Y_theory), 'r--', lw=1.5, label='Theory')
80 axes[0,1].set_title('Phase Verification')
81 axes[0,1].legend(); axes[0,1].grid(True, alpha=0.3)
82
83 axes[1,0].plot(t, x, 'b-', lw=2, label='x(t)')
84 axes[1,0].set_title('Original Signal')
85 axes[1,0].legend(); axes[1,0].grid(True, alpha=0.3)
86
87 axes[1,1].plot(f, np.abs(X_f), 'b-', lw=2, label='|X(f)|')
88 axes[1,1].plot(f, np.abs(Y_f), 'r--', lw=1.5, label='|Y(f)|')
89 axes[1,1].set_title('Spectrum: Before vs After')
90 axes[1,1].legend(); axes[1,1].grid(True, alpha=0.3)
91
92 plt.tight_layout(); plt.show()

```

6 PROBLEM 4: HIDDEN FREQUENCY DISCOVERY

Concept

Complicated signal = sum of simple sine waves. CFT reveals the hidden frequencies as spikes in $|X(f)|$. Reconstruct from found frequencies and verify with MSE.

6.1 Template 16: Frequency Discovery Full Solution

Complete Frequency Discovery Template

```

1 class SignalAnalyzer:
2     def __init__(self, t, f):
3         self.t = t; self.f = f
4
5     def original_signal(self):
6         """The given complicated function."""
7         t = self.t
8         return 2*np.sin(14*np.pi*t) \
9             - np.sin(2*np.pi*t) \
10            * (4*np.sin(2*np.pi*t)*np.sin(14*np.pi*t) - 1)
11        # Simplifies to: sin(2*pi*t) + sin(10*pi*t) + sin(18*pi*
12            t)
13
14    def compute_cft(self, signal):
15        X = np.zeros(len(self.f), dtype=complex)
16        for i, fi in enumerate(self.f):

```

```

16         kernel = np.cos(2*np.pi*fi*self.t) \
17             - 1j*np.sin(2*np.pi*fi*self.t)
18         X[i] = np.trapezoid(signal * kernel, self.t)
19     return X
20
21 def find_frequencies(self, X, threshold=0.5):
22     """Find peaks in |X(f)|      these are the hidden
23         frequencies."""
24     mag = np.abs(X)
25     freqs = []
26     for i in range(1, len(mag)-1):
27         is_peak = mag[i] > mag[i-1] and mag[i] > mag[i+1]
28         big_enough = mag[i] > threshold
29         positive = self.f[i] > 0 # only positive side
30         if is_peak and big_enough and positive:
31             freqs.append(round(self.f[i], 1))
32     return freqs # expected: [1.0, 5.0, 9.0]
33
34 def reconstruct(self, frequencies):
35     """Rebuild signal: amplitude=1, phase=0 (given in
36         problem)."""
37     result = np.zeros(len(self.t))
38     for freq in frequencies:
39         result += np.sin(2 * np.pi * freq * self.t)
40     return result
41
42 def mse(self, a, b):
43     return np.mean((a - b)**2)
44
45 def run(self):
46     ft = self.original_signal()
47     X = self.compute_cft(ft)
48     freqs = self.find_frequencies(X, threshold=0.5)
49     ft_recon = self.reconstruct(freqs)
50     error = self.mse(ft, ft_recon)
51
52     print(f"Found frequencies : {freqs}")
53     print(f"Expected          : [1.0, 5.0, 9.0]")
54     print(f"MSE              : {error:.6e}")
55
56     # Plot
57     fig, axes = plt.subplots(2, 2, figsize=(13, 9))
58
59     axes[0,0].plot(self.t, ft, 'b-', lw=1.5)
60     axes[0,0].set_title('Original f(t)')
61     axes[0,0].set_xlabel('Time t'); axes[0,0].grid(True,
62         alpha=0.3)
63
64     axes[0,1].plot(self.f, np.abs(X), 'b-', lw=1.5)
65     for freq in freqs:
66         idx = np.argmin(np.abs(self.f - freq))
67         axes[0,1].plot(freq, np.abs(X[idx]), 'ro',
68             markersize=10, label=f'f={freq}Hz')
69     axes[0,1].set_title('|X(f)|      Spikes = Hidden
70         Frequencies')
71     axes[0,1].legend(); axes[0,1].grid(True, alpha=0.3)

```

```

68
69     colors = ['r','g','m']
70     for i, freq in enumerate(freqs):
71         axes[1,0].plot(self.t, np.sin(2*np.pi*freq*self.t),
72                        color=colors[i], lw=1.5,
73                        label=f'sin(2pi*{freq}*t)')
74     axes[1,0].set_title('Individual Components')
75     axes[1,0].legend(); axes[1,0].grid(True, alpha=0.3)
76
77     axes[1,1].plot(self.t, ft, 'b-', lw=3, alpha=0.6,
78                   label='Original')
79     axes[1,1].plot(self.t, ft_recon, 'r--', lw=1.5,
80                   label='Reconstructed')
81     axes[1,1].set_title(f'Verification (MSE={error:.2e})')
82     axes[1,1].legend(); axes[1,1].grid(True, alpha=0.3)
83
84     plt.tight_layout(); plt.show()
85
86     #      Run
87     t      = np.linspace(-2, 2, 5000)
88     f      = np.linspace(-15, 15, 3000)
89     analyzer = SignalAnalyzer(t, f)
90     analyzer.run()

```

7 PROBLEM 5: PARSEVAL'S THEOREM

Formula

$$\int_{-\infty}^{\infty} |f(t)|^2 dt = \int_{-\infty}^{\infty} |F(f)|^2 df$$

7.1 Template 17: Parseval's Full Solution

Complete Parseval Template

```

1  class ParsevalVerifier:
2      def __init__(self, t, f):
3          self.t = t; self.f = f
4
5      def signal(self):
6          """Piecewise function over [-3, 3]."""
7          t = self.t
8          return np.where(
9              np.abs(t) <= 1,
10             1 - t**2,                # middle parabola
11             np.where(np.abs(t) <= 3,
12                     (np.abs(t)-3)**2, # side parabolas
13                     0.0))
14
15     def compute_cft(self, ft):
16         X = np.zeros(len(self.f), dtype=complex)

```

```

17         for i, fi in enumerate(self.f):
18             kernel = np.cos(2*np.pi*fi*self.t) \
19                     - 1j*np.sin(2*np.pi*fi*self.t)
20             X[i] = np.trapezoid(ft * kernel, self.t)
21         return X
22
23     def compute_ifft(self, X):
24         """Inverse CFT positive sign."""
25         ft = np.zeros(len(self.t), dtype=complex)
26         for i, ti in enumerate(self.t):
27             kernel = np.cos(2*np.pi*self.f*ti) \
28                     + 1j*np.sin(2*np.pi*self.f*ti) # POSITIVE!
29             ft[i] = np.trapezoid(X * kernel, self.f)
30         return np.real(ft)
31
32     def parseval(self, ft, X):
33         """Compute energy in both domains."""
34         E_time = np.trapezoid(np.abs(ft)**2, self.t)
35         E_freq = np.trapezoid(np.abs(X)**2, self.f)
36         return E_time, E_freq
37
38     def run(self):
39         ft = self.signal()
40         X = self.compute_cft(ft)
41         ft_back = self.compute_ifft(X)
42         E_t, E_f = self.parseval(ft, X)
43         mse_recon = np.mean((ft - ft_back)**2)
44
45         print(f"Energy (time) : {E_t:.6f}")
46         print(f"Energy (freq) : {E_f:.6f}")
47         print(f"Difference : {abs(E_t-E_f):.2e}")
48         print(f"Reconstruction MSE: {mse_recon:.2e}")
49         if abs(E_t - E_f) < 1e-3:
50             print("Parseval VERIFIED!")
51
52         # Plot
53         fig, axes = plt.subplots(2, 2, figsize=(13, 9))
54
55         axes[0,0].plot(self.t, ft, 'b-', lw=2)
56         axes[0,0].set_title('Original Piecewise Signal')
57         axes[0,0].grid(True, alpha=0.3)
58
59         axes[0,1].plot(self.f, np.abs(X), 'r-', lw=1.5)
60         axes[0,1].set_title('Magnitude Spectrum |X(f)|')
61         axes[0,1].grid(True, alpha=0.3)
62
63         axes[1,0].plot(self.t, ft, 'b-', lw=3, alpha=0.6,
64                        label='Original')
65         axes[1,0].plot(self.t, ft_back, 'r--', lw=1.5,
66                        label='IFT Reconstructed')
67         axes[1,0].set_title(f'Original vs IFT (MSE={mse_recon:.2e})')
68         axes[1,0].legend(); axes[1,0].grid(True, alpha=0.3)
69
70         axes[1,1].bar(['Time\n(E_time)', 'Freq\n(E_freq)'],
71                       [E_t, E_f], color=['blue', 'red'], alpha

```

```

72         =0.7)
73     axes[1,1].set_title("Parseval's Energy Comparison")
74     axes[1,1].grid(True, alpha=0.3)
75
76     plt.tight_layout(); plt.show()
77
78 # Run
79 t = np.linspace(-3, 3, 2000)
80 f = np.linspace(-20, 20, 2000)
81 ParsevalVerifier(t, f).run()

```

8 OFFLINE 1: FOURIER EPICYCLES (FS RE-DRAWER)

What This Does

Takes a 2D shape (SVG file). Represents as complex signal $f(t) = x(t) + jy(t)$. Uses Fourier Series to decompose into rotating circles (epicycles). Each circle: radius = $|C_n|$, starting angle = $\angle C_n$, speed = $n\omega$.

8.1 Template 18: Complete FourierEpicycles Class

Complete Epicycles Template

```

1  import numpy as np
2  from svg_utils import load_svg_path
3  from epicycle_animation import save_outputs
4
5  class FourierEpicycles:
6
7      def __init__(self, t, signal, n_harmonics):
8          """
9          t      : time array [0, ..., T] (closed interval)
10         signal : complex array x(t) + j*y(t)
11         N      : number of harmonics
12         """
13         self.t      = t
14         self.signal = signal
15         self.N      = n_harmonics
16         self.T      = t[-1]          # period = last time
17         self.omega  = 2*np.pi / self.T # fundamental frequency
18         self.coefss = {}             # will store Cn values
19
20     def calculate_cn(self, n):
21         """
22         Cn = (1/T) * integral[ f(t) * e^(-j*n*omega*t) ] dt
23         Same as CFT but:
24         - uses n*omega instead of 2*pi*f
25         - divides by T (Fourier Series, not CFT)

```

```

26         """
27         #  $e^{-jn\omega t} = \cos(n\omega t) - j\sin(n\omega t)$ 
28         complexPart = np.cos(n*self.omega*self.t) \
29             - 1j*np.sin(n*self.omega*self.t)
30         integrand    = self.signal * complexPart
31         cn           = np.trapezoid(integrand, self.t) / self.T
32         return cn
33
34     def calculate_all_coefficients(self):
35         """Compute Cn for n = -N, ..., 0, ..., N"""
36         for n in range(-self.N, self.N+1):
37             self.coeffs[n] = self.calculate_cn(n)
38
39     def approximate(self, t):
40         """
41         Reconstruct shape:  $f_{\text{hat}}(t) = \sum [C_n * e^{+jn\omega t}]$ 
42         POSITIVE sign (opposite of calculate_cn)!
43         Works for scalar t or array t.
44         """
45         result = np.zeros(np.shape(t), dtype=complex)
46         for n in range(-self.N, self.N+1):
47             #  $e^{+jn\omega t} = \cos + j\sin$  [POSITIVE for reconstruction]
48             complexPart = np.cos(n*self.omega*t) \
49                 + 1j*np.sin(n*self.omega*t)
50             result      = result + self.coeffs[n] * complexPart
51         return result
52
53 if __name__ == "__main__":
54     import sys
55     from pathlib import Path
56
57     svg_path      = sys.argv[1]
58     N_HARMONICS   = int(sys.argv[2]) if len(sys.argv) > 2 else
59         150
60     stem          = Path(svg_path).stem
61     comparison_path = sys.argv[3] if len(sys.argv) > 3 \
62         else f"{stem}_comparison.png"
63     gif_path      = sys.argv[4] if len(sys.argv) > 4 \
64         else f"{stem}_epicycles.gif"
65
66     t, z = load_svg_path(svg_path, num_points=1000)
67     fs    = FourierEpicycles(t, z, n_harmonics=N_HARMONICS)
68     fs.calculate_all_coefficients()
69     save_outputs(fs, z, comparison_path, gif_path, num_frames
70         =240)

```

Analysis vs Synthesis Sign Rule

Method	Sign	Purpose
calculate_cn	$e^{-jn\omega t}$ (negative)	Find circles (analysis)
approximate	$e^{+jn\omega t}$ (positive)	Draw shape (synthesis)

9 OFFLINE 2: 2D CFT EDGE DETECTOR

What This Does

Image $\rightarrow$ 2D CFT $\rightarrow$ High-pass filter (remove low freq) $\rightarrow$ 2D IFT $\rightarrow$ Edge map.
 Low freq = smooth areas. High freq = edges. Remove smooth $\rightarrow$ only edges remain.

9.1 Template 19: ContinuousImage Class

Image Loading

```

1  from imageio.v2 import imread
2
3  class ContinuousImage:
4      def __init__(self, image_path):
5          # Load grayscale, normalize to [0,1]
6          self.image = imread(image_path, mode='L').astype(float)
7          self.image = self.image / np.max(self.image)
8          # Spatial coords spanning [-1, 1]
9          self.x = np.linspace(-1, 1, self.image.shape[1])
10         self.y = np.linspace(-1, 1, self.image.shape[0])
11
12     def show(self, title="Image"):
13         plt.imshow(self.image, cmap='gray')
14         plt.title(title); plt.axis('off'); plt.show()

```

9.2 Template 20: CFT2D Class — compute_cft()

2D CFT — Separable Integration

```

1  class CFT2D:
2      def __init__(self, image_obj):
3          self.I = image_obj.image
4          self.x = image_obj.x
5          self.y = image_obj.y
6          dx = self.x[1] - self.x[0]
7          dy = self.y[1] - self.y[0]
8          # Nyquist frequency range
9          self.u = np.linspace(-1/(2*dx), 1/(2*dx), self.I.shape[1])
10         self.v = np.linspace(-1/(2*dy), 1/(2*dy), self.I.shape[0])
11
12     def compute_cft(self):
13         """
14         SEPARABLE approach ( $O(N^3)$  not  $O(N^4)$ ):
15         Step 1: Integrate over x for each (u, y)
16         Step 2: Integrate over y for each (u, v)
17
18         Key formulas:

```

```

19     cos(2pi(ux+vy)) = cos(ux)*cos(vy) - sin(ux)*sin(vy)
20     sin(2pi(ux+vy)) = sin(ux)*cos(vy) + cos(ux)*sin(vy)
21     """
22     u_size = len(self.u); v_size = len(self.v)
23     y_size = len(self.y)
24
25     #           Step 1: Integrate over x
26     # cosKorarJonno[i,j] = integral I(x,y_j)*cos(2pi*u_i*x)
27     #                      dx
28     cosKorarJonno = np.zeros((u_size, y_size))
29     sinKorarJonno = np.zeros((u_size, y_size))
30
31     for i in range(u_size):
32         cos_x = np.cos(2*np.pi*self.u[i]*self.x) # (x_size
33         sin_x = np.sin(2*np.pi*self.u[i]*self.x)
34         # self.I*(x_size,) broadcasts -> (y_size, x_size)
35         # trapezoid axis=1 -> integrate over x -> (y_size,)
36         cosKorarJonno[i,:] = np.trapezoid(
37             self.I*cos_x, self.x, axis=1)
38         sinKorarJonno[i,:] = np.trapezoid(
39             self.I*sin_x, self.x, axis=1)
40
41     #           Step 2: Integrate over y
42     finalF = np.zeros((u_size, v_size)) # Real part
43     imgF    = np.zeros((u_size, v_size)) # Imaginary part
44
45     for j in range(v_size):
46         cos_y = np.cos(2*np.pi*self.v[j]*self.y) # (y_size
47         sin_y = np.sin(2*np.pi*self.v[j]*self.y)
48         # cos(ux+vy) = cosK*cosY - sinK*sinY
49         finalF[:,j] = np.trapezoid(
50             cosKorarJonno*cos_y - sinKorarJonno*sin_y,
51             self.y, axis=1)
52         # Im{F} = -integral[I*sin(ux+vy)]
53         imgF[:,j] = -np.trapezoid(
54             cosKorarJonno*sin_y + sinKorarJonno*cos_y,
55             self.y, axis=1)
56
57     return finalF.T, imgF.T # transpose to match image
58                             shape
59
60 def plot_magnitude(self):
61     real, imag = self.compute_cft()
62     mag        = np.sqrt(real**2 + imag**2)
63     plt.figure()
64     plt.imshow(np.log(1 + mag)) # log scale for visibility
65     plt.title("Log Magnitude Spectrum")
66     plt.colorbar(); plt.show()

```

9.3 Template 21: InverseCFT2D Class

2D Inverse CFT

```

1  class InverseCFT2D:
2      def __init__(self, real, imag, u, v, x, y):
3          self.real = real; self.imag = imag
4          self.u = u; self.v = v
5          self.x = x; self.y = y
6
7      def reconstruct(self):
8          """
9          SAME separable strategy as compute_cft() but:
10         - POSITIVE sign:  $e^{+j2\pi(ux+vy)}$ 
11         - Integrate over u then v (not x then y)
12         """
13         u_size = len(self.u); v_size = len(self.v)
14         x_size = len(self.x); y_size = len(self.y)
15
16         #           Step 1: Integrate over u for each x
17         for_real = np.zeros((v_size, x_size))
18         for_imaginary = np.zeros((v_size, x_size))
19
20         for k in range(x_size):
21             cos_u = np.cos(2*np.pi*self.u*self.x[k]) # (u_size
22             ,)
23             sin_u = np.sin(2*np.pi*self.u*self.x[k])
24             # self.real shape: (v_size, u_size)
25             # cos_u[:,np.newaxis] shape: (u_size,1) for
26             # broadcasting
27             for_real[:,k] = np.trapezoid(
28                 self.real*cos_u[:,np.newaxis]
29                 - self.imag*sin_u[:,np.newaxis],
30                 self.u, axis=0)
31             for_imaginary[:,k] = np.trapezoid(
32                 self.real*sin_u[:,np.newaxis]
33                 + self.imag*cos_u[:,np.newaxis],
34                 self.u, axis=0)
35
36         #           Step 2: Integrate over v for each y
37         result = np.zeros((y_size, x_size))
38         for m in range(y_size):
39             cos_v = np.cos(2*np.pi*self.v*self.y[m]) # (v_size
40             ,)
41             sin_v = np.sin(2*np.pi*self.v*self.y[m])
42             result[m,:] = np.trapezoid(
43                 for_real*cos_v[:,np.newaxis]
44                 - for_imaginary*sin_v[:,np.newaxis],
45                 self.v, axis=0)
46
47         return result # real-valued image

```

9.4 Template 22: Complete Edge Detection Pipeline

Full Edge Detector

```

1  if __name__ == "__main__":
2      import sys
3
4      input_path = sys.argv[1]
5      output_path = sys.argv[2]
6      cutoff = float(sys.argv[3]) if len(sys.argv) > 3 else
          15
7
8      # Step 1: Load image
9      img = ContinuousImage(input_path)
10
11     # Step 2: 2D CFT
12     cft2d = CFT2D(img)
13     real, imag = cft2d.compute_cft()
14
15     # Step 3: High-pass filter (remove low freq = smooth
        areas)
16     filt = FrequencyFilter()
17     real_f, imag_f = filt.high_pass(real, imag, cutoff)
18
19     # Step 4: Inverse 2D CFT
20     icft2d = InverseCFT2D(real_f, imag_f,
21                             cft2d.u, cft2d.v, img.x, img.y)
22     edges = icft2d.reconstruct()
23
24     # Step 5: Post-process and save
25     edge_map = np.abs(edges)
26     if edge_map.max() > 0:
27         edge_map = edge_map / edge_map.max() # normalize
28         edge_map = 1 - edge_map # invert: edges=black, bg=white
29
30     plt.imsave(output_path, edge_map, cmap='gray')
31     print(f"Saved edge map to {output_path}")

```

10 PROBLEM 6: IMAGE DENOISING

Concept

Noise = high frequency. Letter = low frequency. Apply FT row by row. Remove high frequencies (low-pass filter). IFT. See the letter! This is OPPOSITE of edge detection (which uses high-pass filter).

10.1 Template 23: Image Denoising Full Solution

Complete Denoising Template

```

1  class NoisyImageDenoiser:
2
3      def __init__(self, image_path):
4          img      = imread(image_path, mode='L').astype(float)
5          self.image = img / np.max(img)
6          self.rows  = self.image.shape[0]
7          self.cols  = self.image.shape[1]
8          self.freqs = np.arange(self.cols)
9
10     def ft_row(self, row):
11         """1D FT of one row. Same as CFT but for pixel domain."""
12         N      = len(row)
13         x      = np.arange(N)
14         X      = np.zeros(N, dtype=complex)
15         for f in range(N):
16             kernel = np.cos(2*np.pi*f*x/N) \
17                     - 1j*np.sin(2*np.pi*f*x/N)
18             X[f]   = np.trapezoid(row * kernel, x)
19         return X
20
21     def ft_all_rows(self):
22         """Apply 1D FT to every row of the image."""
23         FT = np.zeros((self.rows, self.cols), dtype=complex)
24         for i in range(self.rows):
25             FT[i,:] = self.ft_row(self.image[i,:])
26         return FT
27
28     def lowpass_filter(self, FT, cutoff):
29         """
30         Keep: frequencies 0 to cutoff (letter = low freq)
31         Remove: frequencies cutoff to N-cutoff (noise = high
32                freq)
33         Keep: frequencies N-cutoff to N (mirror)
34         OPPOSITE of high_pass filter!
35         """
36         FT_f = FT.copy()
37         N     = self.cols
38         FT_f[:, cutoff:N-cutoff] = 0    # zero out high freq
39         return FT_f
40
41     def ift_row(self, X):
42         """1D IFT of spectrum. POSITIVE sign."""
43         N      = len(X)
44         row    = np.zeros(N, dtype=complex)
45         for xi in range(N):
46             kernel = np.cos(2*np.pi*self.freqs*xi/N) \
47                     + 1j*np.sin(2*np.pi*self.freqs*xi/N) # +j!
48             row[xi] = np.trapezoid(X * kernel, self.freqs)
49         return np.real(row) / N    # normalize by N
50
51     def ift_all_rows(self, FT_filtered):
52         """Apply IFT to every row to get clean image."""

```

```

52         clean = np.zeros((self.rows, self.cols))
53         for i in range(self.rows):
54             clean[i,:] = self.ift_row(FT_filtered[i,:])
55         return clean
56
57     def run(self, cutoff=30):
58         FT_rows = self.ft_all_rows()
59         FT_filtered = self.lowpass_filter(FT_rows, cutoff)
60         clean = self.ift_all_rows(FT_filtered)
61
62         avg_spectrum = np.mean(np.abs(FT_rows), axis=0)
63
64         # Plot
65         fig, axes = plt.subplots(1, 3, figsize=(15, 5))
66         axes[0].imshow(self.image, cmap='gray')
67         axes[0].set_title('Noisy Image'); axes[0].axis('off')
68
69         axes[1].plot(avg_spectrum)
70         axes[1].set_title('Avg Row Spectrum\n(Spikes = Noise)')
71         axes[1].set_xlabel('Frequency'); axes[1].grid(True, alpha
72             =0.3)
73
74         axes[2].imshow(clean, cmap='gray')
75         axes[2].set_title('Denoised (Secret Letter!)')
76         axes[2].axis('off')
77
78         plt.tight_layout(); plt.show()
79         return clean
80
81 if __name__ == "__main__":
82     import sys
83     image_path = sys.argv[1] if len(sys.argv) > 1 else "
84         noisy_image.png"
85     cutoff = int(sys.argv[2]) if len(sys.argv) > 2 else 30
86     NoisyImageDenoiser(image_path).run(cutoff=cutoff)

```

11 QUICK REFERENCE CARDS

11.1 Template 24: CFT vs IFT vs Fourier Series

Sign Rules — Never Confuse These

Transform	Kernel	Sign	Divide by T?
Fourier Series (C_n)	$e^{-jn\omega t}$	Negative	YES ($\div T$)
CFT Forward	$e^{-j2\pi ft}$	Negative	NO
CFT Inverse	$e^{+j2\pi ft}$	Positive	NO
Epicycles C_n	$e^{-jn\omega t}$	Negative	YES ($\div T$)
Epicycles reconstruct	$e^{+jn\omega t}$	Positive	NO
2D CFT forward	$e^{-j2\pi(ux+vy)}$	Negative	NO
2D CFT inverse	$e^{+j2\pi(ux+vy)}$	Positive	NO

11.2 Template 25: All Properties Summary

CFT Properties

Time Shift: $\mathcal{F}\{x(t - t_0)\} = X(f) e^{-j2\pi f t_0}$

Differentiation: $\mathcal{F}\left\{\frac{d^n x}{dt^n}\right\} = (j2\pi f)^n X(f)$

Scaling: $\mathcal{F}\{x(at)\} = \frac{1}{|a|} X\left(\frac{f}{a}\right)$

Modulation: $\mathcal{F}\{x(t)e^{j2\pi f_0 t}\} = X(f - f_0)$

Combined: $\mathcal{F}\{x(at)e^{j2\pi f_0 t}\} = \frac{1}{|a|} X\left(\frac{f - f_0}{a}\right)$

Parseval: $\int |f(t)|^2 dt = \int |F(f)|^2 df$

11.3 Template 26: MSE Interpretation

How to Interpret MSE Values

MSE Value	Meaning	Conclusion
$< 10^{-8}$	Essentially zero	Perfect match
10^{-8} to 10^{-4}	Very small	Property verified
10^{-4} to 10^{-2}	Small	Acceptable
$> 10^{-2}$	Large	Something wrong

11.4 Template 27: 2D CFT vs 2D IFT Summary

2D CFT vs 2D IFT Differences

Aspect	compute_cft()	reconstruct()
Direction	Forward	Inverse
Sign	Negative $(-j)$	Positive $(+j)$
Input	Image $I(x, y)$	Spectrum $F(u, v)$
Output	Spectrum $F(u, v)$	Image $I(x, y)$
Step 1 loop	Over u , integrate x	Over x , integrate u
Step 2 loop	Over v , integrate y	Over y , integrate v
Im{ } sign	Negative at front	No negative
Intermediate	cosK, sinK $(u \times y)$	for real $(v \times x)$

11.5 Template 28: Common Mistakes

Never Do These

- **Never** use `np.fft`, `scipy.fft` or any built-in FT
- **Always** use `np.trapezoid()` for integration
- **Never** manually shift arrays for time-shift (use OOP method)
- **Never** use 4 nested loops for 2D CFT (use separable)
- **Remember:** Forward CFT $\rightarrow$ negative sign
- **Remember:** Inverse IFT $\rightarrow$ positive sign
- **Remember:** Fourier Series divides by T , CFT does not
- **For magnitude plot:** use `np.log(1 + magnitude)`, not `np.log(magnitude)`